

SZEGEDI TUDOMÁNYEGYETEM
Természettudományi és Informatikai Kar

SZAKDOLGOZAT

HealthyEat – Egészséges étkezést támogató webalkalmazás

Készítette:

Bátai Zsolt

programtervező informatikus BSc hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi docens

Szeged, 2026

Feladatkiírás

Az elmúlt időszakban egyre nagyobb figyelmet fordítok az egészséges életmódra és táplálkozásra, és azt tapasztalom, hogy sok ember bizonytalan abban, hogyan kezdjen hozzá a tudatosabb étkezéshez. A különböző trendek, divatdiéták és sokszor ellentmondásos információk mellett nehéz eligazodni, így gyakran hiányzik az a hiteles, személyre szabott támogatás, amely valóban segítené az egyéni célok elérését.

Ezért szeretnék egy olyan alkalmazást készíteni, amely támogatást nyújt a felhasználóknak abban, hogy egészségesebb étkezési szokásokat alakítsanak ki, és ezekhez hosszú távon is hűek maradjanak. A célcsoport elsősorban fiatal felnőttekből és tudatos életmódot kereső érdeklődőkből áll, de az alkalmazás bárki számára hasznos lehet, aki szeretne többet megtudni a táplálkozásról, kipróbálni különböző étrendeket, vagy követni saját kalória- és tápanyagbevitelét.

Az alkalmazás célja, hogy ne csupán egy praktikus étrendtervező eszköz legyen, hanem egy olyan platform, amely információt, motivációt és inspirációt is nyújt. Ez személyre szabott étrendekkel, receptajánlásokkal, kulturális érdekességekkel és táplálkozási tanácsokkal valósul meg – mindezt egy modern, felhasználóbarát felületen.

Tartalmi összefoglaló

Téma megnevezése:

HealthyEat – Egészséges étkezést támogató webalkalmazás

A megadott feladat megfogalmazása:

Egy olyan webalkalmazás fejlesztése, amelyen a felhasználók személyre szabott kalóriaszükségletük alapján recepteket böngészhetnek, napi étrendtervet generálhatnak, étkezésnaplót vezethetnek, bevásárlólistát állíthatnak össze és grafikusán követhetik haladásukat – egyetlen összefüggő felületen.

Megoldási mód:

Háromrétegű webalkalmazás-architektúra: React 18 + Vite 5 SPA frontend, Node.js + Express REST API backend, MySQL 8.0 adatbázis. A személyre szabott kalóriaszükséglet a Mifflin–St Jeor-formulával számítódik. Az étrendterv-generálás Fisher-Yates shuffle-t és diétatípus-alapú szűrést alkalmaz. A tápanyagszámítás egy élelmiszer-adatbázis és háromszintű fuzzy-keresés alapján, futásidőben zajlik. A biztonságot JWT-hitelesítés, bcrypt jelszókezelés és szerepkör-alapú hozzáférés-vezérlés biztosítja.

Alkalmazott eszközök:

React 18, Vite 5, Node.js 18, Express 4, MySQL 8.0, mysql2, jsonwebtoken, bcrypt, Recharts, OpenAI SDK. Fejlesztői környezet: Visual Studio Code. Verziókezelés: Git. Az adatbázis-séma 9 inkrementális SQL-migrációs fájlal verziózott.

Elért eredmények:

Az alkalmazás megvalósítja az összes tervezett funkciót: regisztráció, profilkezelés, TDEE-számítás, receptböngészés, étrendterv-generálás, étkezésnapló, bevásárlólista és haladáskövetési grafikonok. A kézi funkcionális tesztesetek 100%-os sikerességgel zárultak. Az alkalmazás asztali gépen, tableten és mobilon egyaránt kényelmesen használható.

Tartalomjegyzék

Feladatkiírás	1
Tartalmi összefoglaló	2
Tartalomjegyzék	3
1. Bevezetés	5
1.1 Motiváció és problémaháttér	5
1.2 Célkitűzések	6
1.3 A fő felhasználói munkafolyamat	6
1.4 Az MVP-határ meghatározása	7
1.5 A dolgozat felépítése	7
2. Területi áttekintés és piaci elemzés	8
2.1 A digitális táplálkozástámogatás helyzete	8
2.2 Négy főbb megoldás elemzése	8
MyFitnessPal	8
Yazio	8
Lifesum	8
Mealime	9
2.3 Azonosított korlátok és a HealthyEat motivációja	9
3. Követelmények elemzése	10
3.1 Szereplők	10
3.2 Funkcionális követelmények	10
3.3 Nem funkcionális követelmények	11
3.4 Biztonsági követelmények	11
3.5 Követelménymátrix	11
4. Felhasználói esetek	13
4.1 Áttekintés	13
4.2 Autentikáció és profil (UC-01–04)	13
UC-01 – Regisztráció	13
UC-04 – Profil beállítása és TDEE-számítás	14
4.3 Receptkezelés (UC-05–10)	14
4.4 Étrendtervezés és naplózás (UC-11–15)	15
UC-11 – Étrendterv generálása	15
UC-12–15 – Étkezésnapló és dashboard	15
4.5 Bevásárlólista (UC-16–17)	16
5. Rendszertervezés	17
5.1 Háromrétegű architektúra	17
5.2 Frontend felépítése	17
5.3 Backend felépítése	18

5.4 Hitelesítés és jogosultságkezelés	18
5.5 Technológiai döntések	19
5.6 Fontosabb architekturális döntések	20
SPA vs. szerver oldali renderelés.....	20
JWT localStorage-ban vs. httpOnly cookie	20
Snapshot-alapú meal_items.....	20
Állapot nélküli tervező és bevásárlólista.....	20
6. Adatmodell.....	21
6.1 Áttekintés	21
6.2 A főbb entitások	21
users – Felhasználó.....	21
recipes – Recept	22
meals és meal_items – Étkezésnapló	22
foods – Tápanyag-adatbázis	22
comments – Polimorf megjegyzés.....	22
6.3 Kapcsolatrendszer	22
6.4 Adatfolyamok	23
Étkezés naplózása receptből	23
Napi összesítő lekérdezése	23
7. Megvalósítás	24
7.1 Moduláris felépítés	24
7.2 TDEE-számítás – frontend-backend tükrözés.....	24
7.2.1 A TDEE-számítás - implementáció	25
7.3 Étrendtervező algoritmus.....	26
7.4 Bevásárlólista-generálás.....	27
7.5 Chatbot integráció (AI táplálkozási asszisztens).....	28
7.6 REST API végpontok	28
7.7 Hibakezelés és validáció	29
8. Felhasználói felület és UX	30
8.1 Navigációs struktúra	30
8.2 Főbb oldalak.....	30
Nyitóoldal (HomePage)	30
Profiloldal (ProfilePage)	31
Recept adatlap (RecipeDetailPage).....	32
Étkezésnapló (MealLogPage)	33
Haladáskövetés (ProgressPage).....	34
8.3 Reszponzív design.....	35
8.4 Hibás és üres állapotok	36
9. Tesztelés és validáció	37
9.1 Tesztelési stratégia	37
9.2 Funkcionális tesztesetek.....	37
9.3 TDEE-kalkulátor numerikus verifikációja.....	38
Normál eset – férfi, fogyási cél	38

Minimumszint-korlátozás – nő, nagyon alacsony bevitel	38
9.4 Tápanyagszámítás tesztelése	38
9.5 Jogosultságkezelés tesztelése	39
9.6 A tesztelés korlátai	39
9.7 Korlátok és kockázatok	39
10. Összefoglalás és jövőbeli fejlesztések	40
10.1 Elért eredmények	40
10.2 Fejlesztési tanulságok	40
10.3 Jövőbeli bővítési lehetőségek	40
Irodalomjegyzék	42
Nyilatkozatok	43
Nyilatkozat	43
Mesterséges intelligencia használatának nyilatkozata	43
Alkalmazott eszközök	43
Elektronikus melléklet	44

1. Bevezetés

1.1 Motiváció és problémaháttér

Manapság rengeteg ember szeretne tudatosabban étkezni, de a valóságban ez sokszor elmarad – nem feltétlenül motiváció hiányából, hanem mert az ehhez szükséges eszközök szét vannak szórva. Van egy alkalmazás, ahol recepteket keres az ember, egy másik, ahol kalóriákat számol, és egy harmadik, ahol a bevásárlólistát kezeli. Ezeket manuálisan kell összerakosgatni, ami időigényes és elkerülhetetlenül hibák forrása.

A Magyar Táplálkozástudományi Társaság 2022-es adatai szerint a felnőtt magyar lakosság közel 65%-a túlsúlyos vagy elhízott [1]. Hollis és munkatársai (2008) kutatása kimutatta, hogy a rendszeres étkezési napló vezetése önmagában is szignifikánsan javítja a hosszú távú diétás eredményeket [2]. Egy olyan eszköz tehát, amely ezt a folyamatot egyszerűvé és egységessé teszi, valós igényt elégít ki.

A HealthyEat webalkalmazás fejlesztésének célja pontosan ez: összefogni a receptböngészést, az étrendtervezést, az étkezésnaplót és a bevásárlólistát egyetlen, összefüggő felületen. Nem

egy újabb különálló eszköz, hanem egy olyan rendszer, ahol az egyes funkciók egymásra épülnek és adatot adnak át egymásnak.

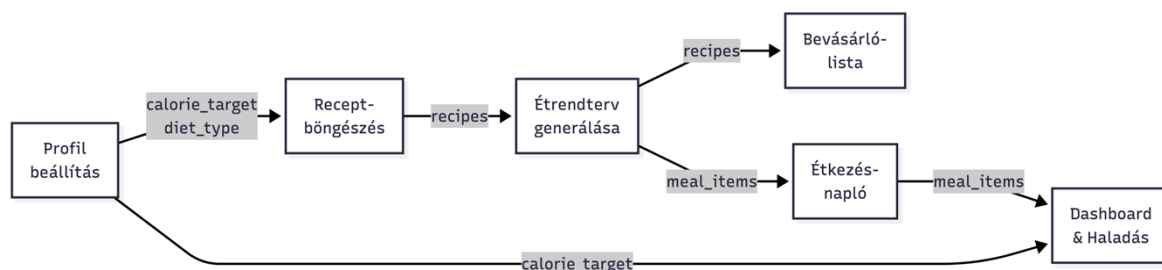
1.2 Célkitűzések

A rendszer személyre szabott kalóriaszámítást végez a Mifflin–St Jeor-formula alapján, figyelembe véve a testtömeget, magasságot, kort, nemet, fizikai aktivitást és az étkezési célt (fogyás, szinten tartás, izomtömeg-növelés). Ez az érték aztán minden modulban – az étrendtervezőben, a dashboardon és a haladáskövetésben – referenciapontként szolgál.

A projekt legfontosabb mérnöki értéke nem egyetlen funkció bonyolultságában rejlik, hanem abban, hogy az egyes modulok összefüggő, végigvihető folyamatot alkotnak: a felhasználó a profilbeállítástól egységes munkafolyamaton keresztül jut el a bevásárlólistáig és a haladás vizualizálásáig, anélkül, hogy bármit manuálisan kellene összekötnie.

1.3 A fő felhasználói munkafolyamat

Az alkalmazás fő értékajánlata egy megszakítás nélküli folyamatban ölt testet. A felhasználó először beállítja a profilját – megadja a biometrikus adatait és az étkezési célját –, amire a rendszer kiszámítja a napi kalóriacélt. Ezután recepteket böngész, és egy gombnyomással étrendtervet generál: a rendszer a profiljának megfelelő recepteket rendel a négy étkezési napszakhoz. A tervből automatikusan bevásárlólista generálható. Vásárlás és főzés után az elfogyasztott ételeket rögzíti az étkezésnaplóban, a dashboard pedig azonnal megmutatja, hol



1.1. ábra: Fő felhasználói munkamenet.

tart a napi kalóriacéljához képest. A haladáskövetési oldalon heti és havi grafikon szemlélteti a hosszabb távú trendet.

1.4 Az MVP-határ meghatározása

A fejlesztési hatókör meghatározásához az MVP (Minimum Viable Product) módszertant alkalmaztam. Az MVP azt a legkisebb koherens funkcióhalmazt jelenti, amely már önmagában értelmes és végigvihető felhasználói élményt nyújt.

Az MVP-be biztosan beletartozó funkciók: regisztráció és JWT-alapú bejelentkezés, felhasználói profil és TDEE-számítás, receptböngészés és keresés, étrendterv-generálás, étkezésnapló és napi összesítő dashboard, bevásárlólista-generálás és haladáskövetési grafikonok. Részben megvalósított funkciók: receptértékelés és kommentek, oktatási cikkek, admin panel (moderálás). Opcionális kiegészítő: AI-alapú táplálkozási chatbot (OpenAI gpt-4o-mini).

Fontos kiemelni az admin modul korlátait: a controller-implementáció TODO-jelzéssel rendelkezik, és az `admin.routes.js`-ből hiányzó `authenticate + authorize` middleware-lánc biztonsági kockázatot jelent, amelyet a jövőbeli fejlesztés során pótolni szükséges. Szintén tudatos döntés, hogy automatizált tesztsuite nem készült az MVP keretében – a kézi tesztelés lefedi a főbb use case-eket, a regressziótesztelés a jövőbeli fejlesztés feladata.

1.5 A dolgozat felépítése

A 2. fejezet piackutatást végez: elemzi a négy legjelentősebb hasonló alkalmazást, azonosítja korlátaikat, és ebből vezeti le a HealthyEat fejlesztési motivációját. A 3. fejezet a funkcionális és nem funkcionális követelményeket részletezi. A 4. fejezet a felhasználói eseteket (UC-01–23) írja le. Az 5. fejezet a háromrétegű rendszerarchitektúrát mutatja be, beleértve a főbb architekturális döntések indoklását. A 6. fejezet az adatmodellt tárgyalja. A 7. fejezet a megvalósítás részleteit tartalmazza: modulok, REST API, hibakezelés, validáció. A 8. fejezet a felhasználói felületet és a UX-elveket mutatja be. A 9. fejezet a tesztelési eredményeket foglalja össze. A 10. fejezet összegez és jövőbeli irányokat jelöl meg. Az melléklet a MI-használati nyilatkozatot tartalmazza.

2. Területi áttekintés és piaci elemzés

2.1 A digitális táplálkozástámogatás helyzete

A táplálkozástámogató alkalmazások piaca érett és versengő. Az elérhető megoldások négy fő kategóriába sorolhatók: kalórianapló és makrökövetők (MyFitnessPal, Yazio, Cronometer), étrendtervezők (Mealime, PlateJoy), receptajánlók (Yummly) és komplex életmód-platformok (Lifesum, Noom). A HealthyEat az első két kategóriát ötvözi: receptbank, TDEE-alapú étrendtervező és étkezésnapló egy egységes webalkalmazásban.

A piacon elérhető alkalmazások elsősorban általános célú, kereskedelmi termékek, amelyek korlátozottan testreszabhatók és belső működésük nem módosítható. A dolgozat keretében fejlesztett rendszer ezzel szemben kifejezetten a meghatározott funkcionális igényekre készült, ezért rugalmasan bővíthető és továbbfejleszthető.

2.2 Négy főbb megoldás elemzése

MyFitnessPal

A MyFitnessPal (Under Armour Inc., 2005) a világ legelterjedtebb kalóriaszámláló alkalmazása, több mint 200 millió regisztrált felhasználóval. Óriási erőssége a 14 milliós élelmiszer-adatbázis és a vonalkód-szkenner. Az alapfunkciók ingyenesek, de a részletes makrökövetés és az étrendtervező prémium előfizetéshez kötött. Kiemelendő gyengesége, hogy a felhasználói felület az évek során egyre zsúfoltabbá vált, és a 2018-as adatszivárgásban 150 millió fiók kompromittálódott.

Yazio

A Yazio (YAZIO GmbH, Berlin, 2013) az EU egyik legnépszerűbb táplálkozástámogató alkalmazása 50 millió letöltéssel. Kiemelkedő funkciója a böjtidőzítő és az integrált receptajánló; érhető el magyarul is.

Lifesum

A Lifesum (Lifesum AB, Stockholm, 2013) egyedi megközelítése, hogy nem csupán kalóriákat követ, hanem egy ún. Life Score minőségi pontszámot is számít. Személyre szabott diétatípus-sablonokat (ketogén, mediterrán, DASH) kínál, és széles körű fitnesz-platform integrációval rendelkezik.

Mealime

A Mealime (Mealime Inc., 2016) étkezéstervezésre specializálódott: diétás preferenciák alapján heti menüt állít össze, majd automatikusan bevásárlólistát generál. Ez utóbbi funkció kiválóan van megoldva – és tanulságos mintát adott a HealthyEat bevásárlólista-moduljához. Hátránya, hogy nincs étkezésnapló vagy haladáskövetés: a rendszer kizárólag tervezésre alkalmas.

Szempont	MyFitnessPal	Yazio	Lifesum	Mealime	HealthyEat
Célcsoport	Általános felnőttek, sportolók	Fogyni kívánó felnőttek, bőjtölők	Életmód-tudatos felhasználók	Elfoglalt családok, főzni vágyók	Tudatos életmódra törekvő felhasználók
Kalórianapló	Igen	Igen	Igen	Nem	Igen
Makrókövetés	Igen	Igen	Igen	Nem	Igen
TDEE-alapú cél	Részben	Igen	Részben	Nem	Igen
Receptböngészés	Részben	Igen	Részben	Igen	Igen
Étrendtervezés	Prémium / korlátozott	Prémium	Prémium	Igen	Igen
Bevásárlólista	Nem / korlátozott	Korlátozott	Nem / korlátozott	Igen	Igen
Haladáskövetés	Igen	Igen	Igen	Nem	Igen
Magyar nyelv	Nem teljes	Igen	Nem teljes	Nem	Igen
Algoritmus átláthatósága	Zárt	Zárt	Zárt	Zárt	Nyílt, dokumentált
Fizetőfal	Több fontos funkció prémium	Tervező és riportok prémium	Teljes funkció prémium	Egyes funkciók prémium	Nincs
Fő erősség	Nagy élelmiszer-adatbázis	Egyszerű célbeállítás, magyar felület	Vizuális visszajelzés, életmódpontszám	Erős menütervezés és bevásárlólista	recept-terv-napló-lista folyamat
Fő hiányosság	tervező nem igazán ingyenes	Zárt algoritmus, prémium AI-tervező	Prémium fókusz, napló és terv külön	Nincs naplózás és haladáskövetés	nincs vonalkód-szkennel
HealthyEat számára levont tanulság	Egyszerű dashboard és napi cél fontos	célbeállítás legyen egyszerű	Motiváló grafikonok hasznosak	közvetlenül kapcsolva a tervhez	Koherens MVP a fő érték

2.1. ábra: Összefoglaló táblázat az áttekintett alkalmazásokról.

2.3 Azonosított korlátok és a HealthyEat motivációja

Az összehasonlítás alapján három rendszerszintű hiányosság rajzolódik ki. Először is az algoritmusok átláthatatlansága: egyik vizsgált alkalmazás sem teszi nyilvánossá a TDEE-számítás részleteit vagy az étrendtervező logikáját. Másodsor: a napló és a tervező szétválasztása – a Mealime kizárólag tervező, a MyFitnessPal elsősorban napló; a kettő integrációja ingyenesen sehol nem elérhető. Harmadsor: a bevásárlólista és az étrendterv csak a Mealime-ban van összekapcsolva, de ott nincs naplózás.

A HealthyEat ezekre a hiányokra kínál választ: az összes kulcsfunkciót, egy helyen, nyílt és auditálható logikával biztosítja. A rendszer nem a piaci vezető megoldások megismétlésére törekszik – hanem egy jól definiált, koherens MVP-re, amelynek minden funkciója szorosan kapcsolódik a többihez.

Összefoglalva: a HealthyEat pontosan azt a piaci rést célozza, amelyet a meglévő megoldások nem fednek le egységesen és ingyenesen. A MyFitnessPal az étkezésnaplózásban erős, a Mealime a tervezésben és bevásárlólistában, a Lifesum a vizuális visszajelzésben – de ezek fizetős, különálló rendszerekben élnek. A HealthyEat értéke az, hogy a tervezés, naplózás, TDEE-alapú kalóriacél, bevásárlólista és haladáskövetés egyetlen összefüggő folyamatba kapcsolódik, átlátható algoritmusokkal.

3. Követelmények elemzése

3.1 Szereplők

A rendszer három típusú felhasználót különböztet meg. A vendég hitelesítés nélkül böngészi a recepteket és olvassa a cikkeket, de személyes funkciókat nem ér el. A regisztrált felhasználó érvényes JWT-tokennel rendelkezik, és hozzáfér az összes személyes funkcióhoz: profil, étrendtervező, étkezésnapló, bevásárlólista, haladáskövetés és chatbot. Az admin a regisztrált felhasználó jogain felül hozzáfér a tartalommoderáláshoz, felhasználókezeléshez és cikk-közzétételhez.

3.2 Funkcionális követelmények

A funkcionális követelmények három prioritási szintbe sorolhatók. A Must (kötelező) követelmények az MVP fő folyamatát alkotják: regisztráció és bejelentkezés, profilkezelés és TDEE-számítás, receptböngészés és keresés, saját receptek létrehozása/szerkesztése/törlése, étrendterv-generálás, étkezésnapló és napi összesítő, bevásárlólista-generálás, heti/havi haladáskövetés. A Should (erősen ajánlott) követelmények a rendszer értékét növelik, de nem kritikusak: receptértékelés és kommentek, oktatási cikkek, admin moderálás. A Could (opcionális) kategóriába egyetlen funkció kerül: az AI-alapú chatbot.

A legfontosabb elfogadási feltételek: a TDEE-számítás eltérése a referencia-értéktől 1 kcal-nál kisebb; az étrendterv kalóriaarányai reggeli 25%, ebéd 35%, vacsora 30%, snack 10% ($\pm 5\%$ toleranciával a receptek diszkrét kalóriaértékei miatt); azonos nevű hozzávalók a bevásárlólistán helyesen aggregálódnak; más felhasználó erőforrásának módosítási kísérlete 403 Forbidden választ kap.

3.3 Nem funkcionális követelmények

Teljesítmény: a receptlista oldal első betöltése 2 másodpercen belül megtörténik; az API átlagos válaszideje 500 ms alatt marad normál terhelésnél. Biztonság: jelszavak bcrypt algoritmussal tárolva (cost factor ≥ 10); JWT minden védett végponton ellenőrzött; titkos értékek (API-kulcs, JWT-titok) soha nem kerülnek verziókezelte forráskódba. Használhatóság: a fő munkafolyamat (regisztrációtól a dashboardig) rendszer-dokumentáció elolvasása nélkül elvégezhető, minden lépés legfeljebb 3 UI-interakcióból áll. Reszponzivitás: a felület 375 px minimális szélességtől asztali felbontásig hibátlanul megjelenik és kezelhető. Reprodukálhatóság: az alkalmazás npm run setup egységes paranccsal feltelepíthető tiszta környezetben, 15 percen belül.

3.4 Biztonsági követelmények

A biztonsági követelmények öt területet fednek le. Jelszókezelés: a password_hash mező soha nem kerül API-válaszba; a user.model.js findById() lekérdezése szándékosan kizárja. JWT-hitelesítés: minden védett végponton middleware-ellenőrzés; érvénytelen vagy lejárt tokenre 401-es válasz. Szerepkör-alapú hozzáférés: admin-only végpontokra user tokennel 403. Tulajdonosi ellenőrzés: recept szerkesztése és törlése esetén a service réteg ellenőrzi, hogy a kérelmező azonos-e a tulajdonossal. SQL-injection-védelem: az összes adatbázis-lekérdezés mysql2 paraméteres lekérdezéssel épül fel, közvetlen string-konkatenáció nincs.

3.5 Követelménymátrix

A követelménymátrix a fejlesztés során folyamatos ellenőrzési eszközként szolgált: minden implementált modul és API-végpont visszakövethető egy vagy több funkcionális követelményhez. Egy funkciót akkor tekintettem elkészültnek, ha az adott követelményhez tartozó backend route és service implementálva van, a frontend oldal megjeleníti és kezeli a választ, és a kézi tesztelés során a várt HTTP-státuszkód érkezett vissza. Státuszok: Elkészült (Must és Should prioritású, teljesen implementált), Részlegesen elkészült (pl. admin panel alapfunkciók), Nem elkészült / kizárt (opcionális és tudatosan elhagyott elemek).

Az alábbi táblázat a funkcionális követelményeket köti össze az érintett komponensekkel.

Funkcionális terület	Backend route	Service	Frontend oldal
Autentikáció (FK-01–06)	/api/auth/*	auth.service.js	LoginPage, RegisterPage
Profil és TDEE (FK-07–10)	/api/users/*	users.service.js	ProfilePage
Receptek (FK-11–18)	/api/recipes/*	recipes.service.js	RecipesPage, RecipeDetailPage
Étrendtervező (FK-19–21)	/api/planner	planner.service.js	DietPlannerPage
Étkezésnapló (FK-22–27)	/api/meals/*	meals.service.js	MealLogPage, DashboardPage
Bevásárlólista (FK-28–30)	/api/shopping	shopping.service.js	ShoppingListPage
Haladáskövetés (FK-31–33)	/api/meals/history	(aggregáció)	ProgressPage
Chatbot (FK-38–40)	/api/chatbot	chatbot.service.js	ChatbotPage

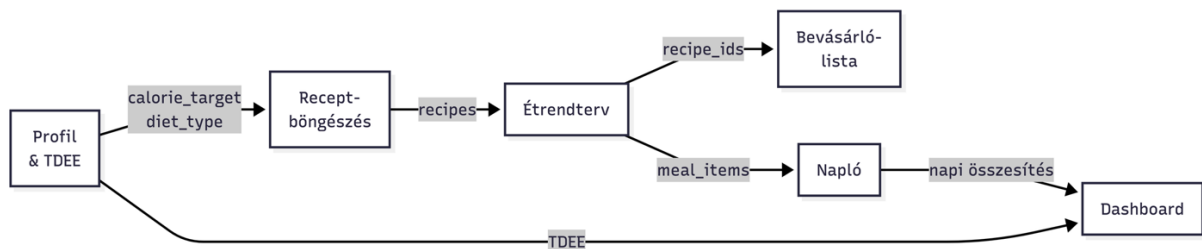
3.1. ábra: Követelménymátrix.

4. Felhasználói esetek

4.1 Áttekintés

A rendszer 23 felhasználói esetet definiál (UC-01–UC-23), amelyek négy fő csoportba szerveződnek: autentikáció és profil (UC-01–04), receptkezelés (UC-05–10), étrendtervezés és naplózás (UC-11–15), valamint bevásárlólista és kiegészítő funkciók (UC-16–23). Az alábbi ábra ezek kapcsolatát mutatja be.

A rendszer fő értéke a teljes étkezési folyamat végigvihető egységességében rejlik. A felhasználói út hat lépésből áll: (1) Regisztráció és profilbeállítás – a felhasználó megadja biometrikus adatait, a rendszer kiszámítja a napi kalóriacélt (TDEE), amely azonnal érvényes minden következő modulban. (2) Receptböngészés – kulcsszóra keresés vagy diétatípus szerinti szűrés; minden recepthez adagonkénti tápanyagadatok. (3) Étrendterv generálása – egy kérésre a rendszer a négy napszakhoz (reggeli 25%, ebéd 35%, vacsora 30%, snack 10%) személyre szabott recepteket rendel. (4) Bevásárlólista – a tervből automatikusan, deduplikált és kategorizált lista generálható. (5) Étkezésnapló – az elfogyasztott ételek rögzítése; a makróértékek automatikusan kitöltődnek. (6) Dashboard és haladáskövetés – napi bevitel vs. kalóriacél; heti és havi grafikon.



4.1. ábra: A fő felhasználói munkafolyamat és az adatkapcsolatok

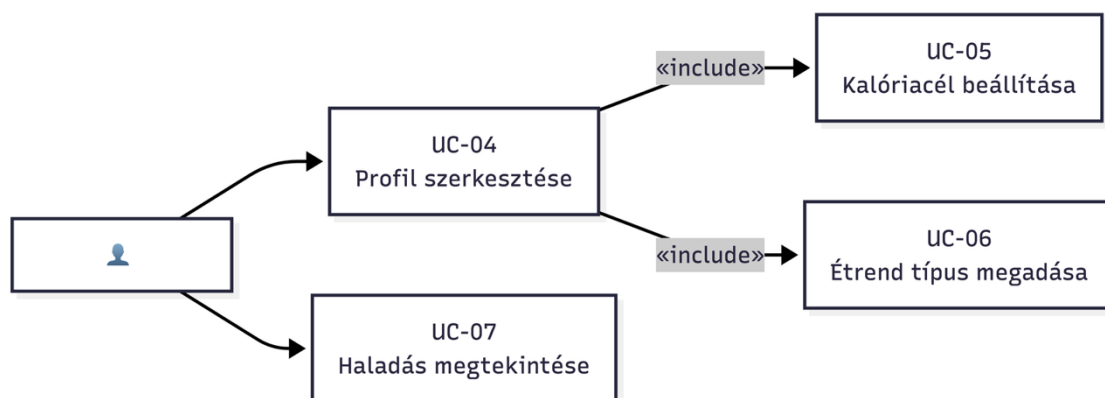
4.2 Autentikáció és profil (UC-01–04)

UC-01 – Regisztráció

A vendég felhasználónév, e-mail cím és jelszó megadásával fiókot hoz létre. A backend ellenőrzi az e-mail és felhasználónév egyediségét (ütközés esetén 409 Conflict), bcrypt algoritmussal hash-eli a jelszót (cost factor 10), elmenti a rekordot, majd JWT-tokent állít ki. Sikeres regisztráció után a rendszer automatikusan bejelentkezteti a felhasználót, és a profiloldalra irányítja, hogy megadhassa biometrikus adatait.

UC-04 – Profil beállítása és TDEE-számítás

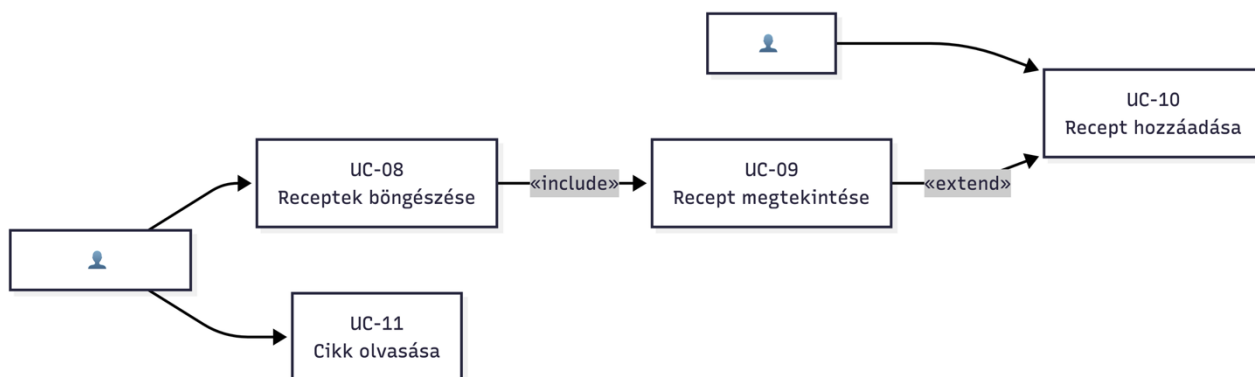
A bejelentkezett felhasználó megadja a testtömegét, magasságát, korát, nemét, aktivitási szintjét és étkezési célját. A rendszer a Mifflin–St Jeor-formula alapján kiszámítja a BMR-t, aktivitási szorzóval megszorozza (TDEE), majd a célkorrekcióval adja meg a végső kalóriacélt (fogyás: –500 kcal, szinten tartás: 0, izomtömeg-növelés: +300 kcal). Biztonsági minimum érvényesül: nők esetén 1200 kcal, férfiak esetén 1500 kcal. Az eredmény azonnal érvényes lesz az étrendtervező és a dashboard számára.



4.2. ábra: Profil beállítás és TDEE-számítás használati eset diagram.

4.3 Receptkezelés (UC-05–10)

A receptek böngészése és keresése (UC-05, UC-06) hitelesítés nélkül is elérhető – a vendég is megtalálhatja a számára releváns recepteket cím alapján keresve vagy diétatípus szerint szűrve. A recept adatlapján (UC-07) megjelennek a hozzávalók, az elkészítési lépések és az adagonkénti tápanyagadatok, amelyeket a rendszer futásidőben számít ki a 55 tétel



4.3. ábra: Receptkezelés használati eset diagram

élelmiszer-adatbázis alapján. Bejelentkezett felhasználók saját receptet hozhatnak létre, szerkeszthetnek és törölhetnek (UC-08–10); más felhasználó receptjének módosítási kísérlete 403-as hibaválaszt kap.

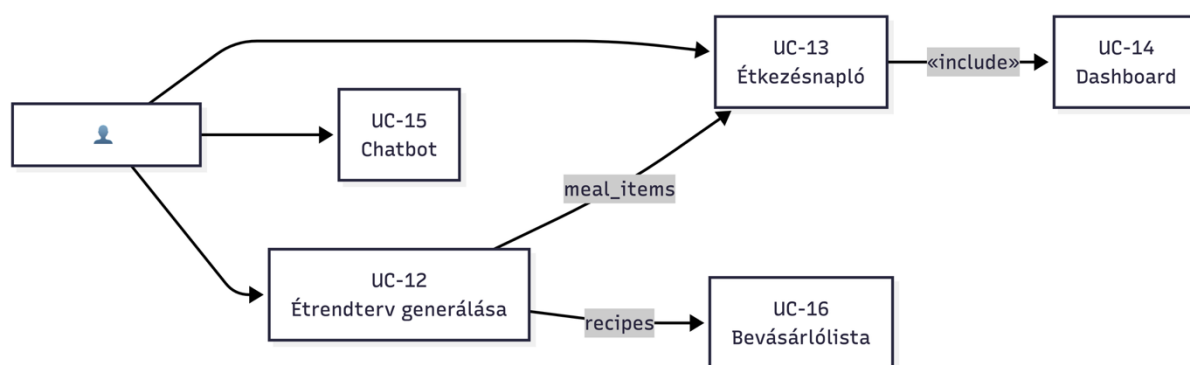
4.4 Étrendtervezés és naplózás (UC-11–15)

UC-11 – Étrendterv generálása

A rendszer a felhasználó `calorie_target` és `diet_type` értékei alapján generál napi tervet. Az algoritmus először a diétatípusnak megfelelő recepteket keresi; ha kevesebb mint 3 találat van, az összes receptre visszaesik. Fisher-Yates shuffle után a négy étkezési slothoz (reggeli 25%, ebéd 35%, vacsora 30%, snack 10%) rendel recepteket. Fontos: a slot kalóriaarányai iránymutatások – az algoritmus nem szűr tényleges receptkalória alapján, mert ez túl szűkre szabná a választékot.

UC-12–15 – Étkezésnapló és dashboard

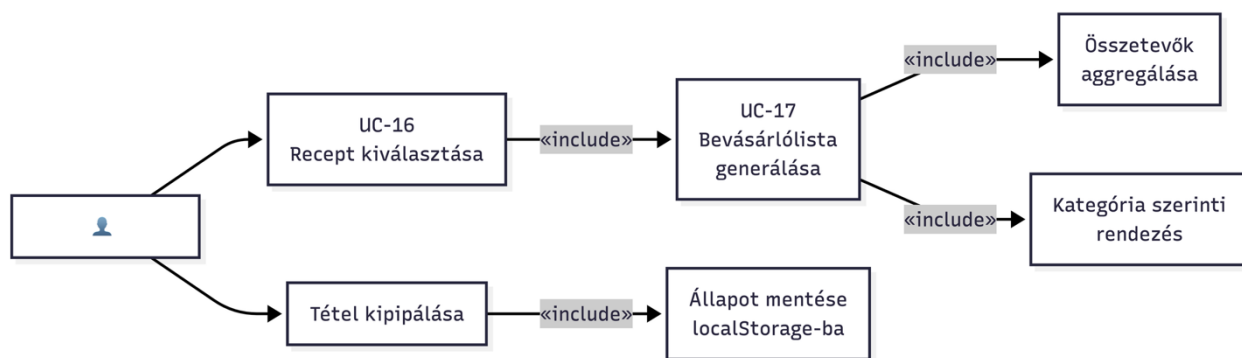
A napló napi struktúrában rögzíti az elfogyasztott ételeket. Egy étkezésbejegyzés (meal) konténer szerepet tölt be – az egyes tételek (meal_items) ehhez kapcsolódnak. A tételek háromféleképpen adhatók hozzá: az élelmiszer-adatbázisból keresve (a makrók automatikusan skálázódnak a megadott mennyiség alapján), receptből naplózva (az adagonkénti értékek számíthatók ki), vagy manuálisan. Minden esetben a makrók a bejegyzés pillanatában mentődnek el – ha a forrásrecept később módosul, a régi naplóbejegyzés nem változik (snapshot-elv). A dashboard ezekből az adatokból számítja a napi összesítőt.



4.4. ábra: Étkezésnapló használati eset diagram

4.5 Bevásárlólista (UC-16–17)

A bevásárlólista-generálás az MVP egyik legegyszerűbb, mégis legértékesebb funkciója. A felhasználó kiválaszt egy vagy több receptet (jellemzően az étrendtervből), és a rendszer automatikusan összesíti a hozzávalókat: az azonos nevű összetevők mennyiségei aggregálódnak, az eredmény kategóriák szerint (Protein, Vegetables, Dairy stb.) rendezve jelenik meg. Minden tétel mellé jelölőnégyzet kerül, amit vásárlás közben lehet kipipálni – az állapot localStorage-ban megőrződik az oldalfrissítés után is.



4.5. ábra: Bevásárló lista használati eset diagram

5. Rendszertervezés

5.1 Háromrétegű architektúra

A HealthyEat klasszikus háromrétegű webalkalmazás-architektúrára épül. A megjelenítési réteget egy React 18 + Vite 5 alapú SPA alkotja, amely böngészőben fut. Az üzleti logika rétegét egy Node.js + Express REST API képviseli, Routes → Controllers → Services → Models szétválasztással. Az adatréteget MySQL 8.0 relációs adatbázis alkotja, mysql2 connection pool kezelőn keresztül. Ez a szétválasztás háromféle előnnyel jár: a frontend cserélhető anélkül, hogy az API-t módosítani kellene (pl. később mobilapp is épülhet rá), az üzleti logika önállóan tesztelhető, és az adatbázis-séma a migrációalapú fejlesztésnek köszönhetően mindig reprodukálható.

5.2 Frontend felépítése

A frontend belépési pontja a main.jsx, amely React.StrictMode-ba és BrowserRouter-be csomagolja az alkalmazást. Az útvonalak az App.jsx-ben vannak definiálva, két csoportban: nyilvános oldalak (főoldal, bejelentkezés, regisztráció, receptlista, cikklista) és védett oldalak, amelyek a ProtectedRoute HOC mögött élnek. Ez utóbbi ellenőrzi, hogy van-e érvényes JWT-token a localStorage-ban; ha nincs, a bejelentkezési oldalra irányít. A frontend egyetlen, megosztott Axios-példányt használ (api.js), amelynek request interceptora automatikusan csatolja az Authorization: Bearer fejléct minden kéréshez, ha van token a localStorage-ban. Ez azt jelenti, hogy az egyes service-fájloknak nem kell törődniük a hitelesítéssel – az teljesen transzparens marad.

```
1  const api = axios.create({
2    baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:5001/api',
3  });
4
5  api.interceptors.request.use((config) => {
6    const token = getToken();
7    if (token) {
8      config.headers.Authorization = `Bearer ${token}`;
9    }
10   return config;
11 });
```

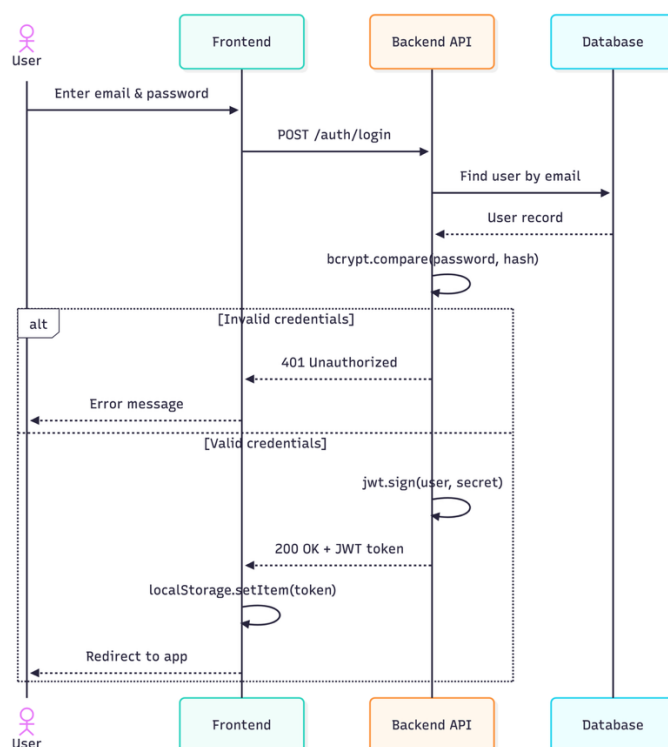
5.1. ábra: Az Axios request interceptor implementációja.

5.3 Backend felépítése

A backend MVC-szerű, négyrétegű struktúrát követ. A Routes réteg csak URL-leképezést és middleware-láncolást tartalmaz. A Controllers réteg a HTTP-réteget kezeli: kinyeri a req.params/body/user értékeket, meghívja a megfelelő Service-t, és visszaadja a HTTP-választ. A Services rétegben él az összes üzleti logika: validáció, jogosultságellenőrzés, számítások, Model-hívások koordinálása. A Models réteg kizárólag paraméteres SQL-lekérdezéseket tartalmaz a mysql2 pool segítségével. A backend 11 route-prefixet kezel: /api/auth, /api/users, /api/recipes, /api/meals, /api/planner, /api/shopping, /api/chatbot, /api/foods, /api/articles, /api/comments, /api/admin és /api/health. A GET-végpontok általában nyilvánosak (vendég is elérheti a recepteket), az írási műveletek Bearer JWT-t igényelnek, az admin végpontok pedig authorize('admin') middleware-rel is védve vannak.

5.4 Hitelesítés és jogosultságkezelés

A rendszer állapotmentes JWT-alapú hitelesítést alkalmaz. Bejelentkezéskor a szerver HS256 algoritmussal, 7 napos érvényességgel állít ki token-t (payload: id, email, role). A token a böngésző localStorage-ában él; az Axios interceptor minden kéréshez automatikusan csatolja.



5.2. ábra: A bejelentkezés szekvencia diagramja.

A backend három middleware-típust definiál. Az `authenticate` kötelező hitelesítést végez: hiányzó vagy érvénytelen token esetén 401-et ad vissza. Az `authenticateOptional` nem blokkolja a kérést, csak ha van token, beállítja a `req.user` értéket (pl. receptértékelések lekérésekor – bejelentkezve a saját értékelés is visszakérül). Az `authorize(...roles)` szerepkör-ellenőrzést végez az `authenticate` után – admin végpontokon user tokennel 403 a válasz.

```
1 function authenticate(req, res, next) {
2   const authHeader = req.headers.authorization;
3
4   if (!authHeader || !authHeader.startsWith('Bearer ')) {
5     return res.status(401).json({ error: 'Authorization token missing.' });
6   }
7
8   const token = authHeader.split(' ')[1];
9
10  try {
11    req.user = jwt.verify(token, process.env.JWT_SECRET);
12    next();
13  } catch {
14    return res.status(401).json({ error: 'Invalid or expired token.' });
15  }
16 }
```

5.3. ábra: Az `authenticate` middleware implementációja.

5.5 Technológiai döntések

A technológiai stack megválasztásakor néhány alapelvet követtem: iparban bevált és jól dokumentált eszközök, aktív közösség, és minél kisebb mismatch a frontend és backend között. Az eredmény egy JavaScript-egységes stack lett: a frontend és a backend ugyanazon a nyelven íródott, ami lehetővé tette egyes segédfüggvények (pl. a TDEE-kalkulátor) megosztását mindkét oldalon.

Frontend: React 18 a komponensalapú UI-hoz (hooks API elegáns állapotkezelést tesz lehetővé globális állapotkönyvtár nélkül), Vite 5 a gyors HMR fejlesztési élményhez, Recharts a diagramokhoz (React-natív, ResponsiveContainer reszponzív megjelenítéshez). Backend: Node.js 18 LTS és Express 4 a minimális, de rugalmas API-hoz; mysql2 a paraméteres lekérdezésekhez és connection pool kezeléshez; jsonwebtoken és bcrypt a biztonsághoz; OpenAI SDK az AI chatbothoz. Adatbázis: MySQL 8.0, mert az adatmodell erősen relációs és a FOREIGN KEY + CASCADE DELETE kényszerfeltételek adatbázis szinten is érvényesíthetők.

5.6 Fontosabb architektúráis döntések

SPA vs. szerver oldali renderelés

React SPA-t választottam Vite-tal, mert az alkalmazás tartalma szinte teljes egészében hitelesítéshez kötött – ezért SSR nem nyújtana SEO-előnyt. Az SPA tiszta szétválasztást tesz lehetővé az API és a megjelenítési réteg között, és a frontend önállóan deployolható statikus fájlként.

JWT localStorage-ban vs. httpOnly cookie

JWT tokenet localStorage-ban tárolom, mert ez alacsonyabb komplexitású MVP-szinten. A kompromisszum ismert: JavaScript-ből olvasható token XSS-sebezhetőséget jelent. Teljesebb biztonsághoz httpOnly cookie ajánlott, de ez CSRF-védelem implementálását is megkövetelné.

Snapshot-alapú meal_items

A meal_items tábla közvetlenül tárolja a makrókat a bejegyzés pillanatában, nem hivatkozik vissza receptre idegen kulccsal. Ha a forrásrecept utólag módosul, a régi naplóbejegyzés változatlan marad – ez az étkezési naplóban alapvető elvárás. A denormalizáció árát (redundáns adatok) az MVP méreténél bőven megéri a historikus integritás.

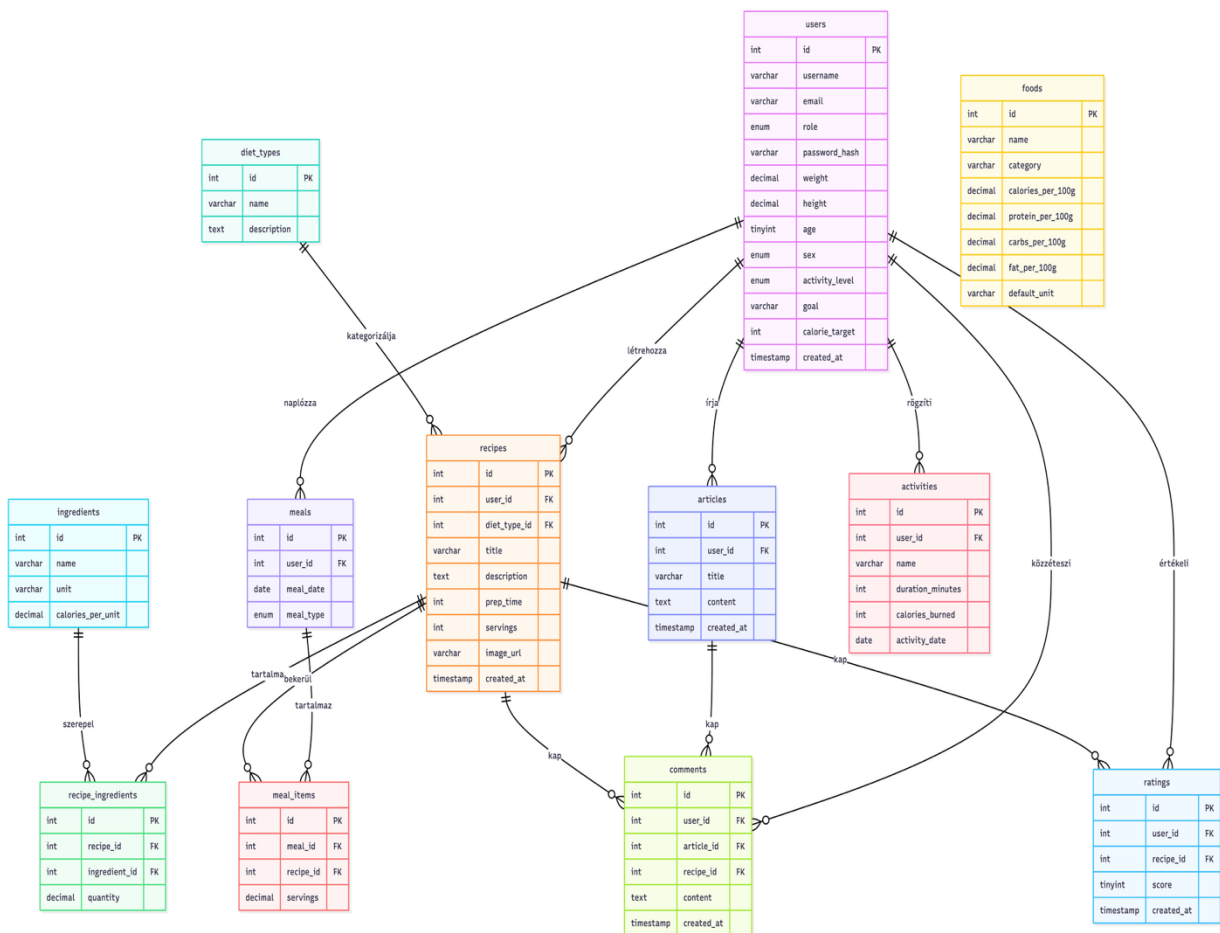
Állapot nélküli tervező és bevásárlólista

Az étrendterv és a bevásárlólista generálása nem kerül adatbázisba – minden kéréskor újraszámítódnak. Ez csökkenti a séma komplexitását, és az MVP-ben nincs igény a korábbi tervek visszakeresésére. Jövőbeli fejlesztésként egy weekly_plans tábla hozzáadásával ez könnyen megoldható.

6. Adatmodell

6.1 Áttekintés

Az adatokat MySQL 8.0 relációs adatbázis tárolja a healthy_eating_app sémában, utf8mb4_unicode_ci karakterkészlettel. Az adatmodell 12 táblából épül fel; a séma az alap schema.sql és 9 inkrementális migrációs fájl együttes alkalmazásával jön létre. Ez utóbbi azt jelenti, hogy az adatbázis bármely tiszta MySQL 8 példányon egyetlen paranccsal reprodukálható.



6.1. ábra: Entitás-kapcsolati diagram.

6.2 A főbb entitások

users – Felhasználó

A rendszer gyökérentitása. Tárolja az azonosítási adatokat (username, email, password_hash, role), a biometrikus adatokat (weight, height, age, sex, activity_level) és a TDEE-számítás eredményét (calorie_target, diet_type, goal). A password_hash mező soha nem kerül API-válaszba – a user.model.js findById() lekérdezése szándékosan kizárja a SELECT-ből.

recipes – Recept

A receptkatalógus egységei. Az ingredients mező JSON-kódolt sztringtömböt tartalmaz szöveges hozzávalósorokat (pl. ["200g csirkemell", "1 fej hagyma"]) – a tápanyagszámítás futásidőben, karakterlánc-feldolgozással történik a foods tábla alapján.

meals és meal_items – Étkezésnapló

A meals tábla egy étkezési alkalmat jelöl (dátum + típus: breakfast/lunch/dinner/snack). A meal_items tábla tartalmazza a tényleges tételeket a bejegyzés pillanatában érvényes makrókkal (snapshot-elv). A két tábla között CASCADE DELETE kapcsolat van: ha egy meals sor törlődik, a hozzá tartozó tételek is törlődnek.

foods – Tápanyag-adatbázis

55 alapélelmiszert tartalmaz 100 grammra vetített makrotápanyag-adatokkal. Ez a tábla a recipeNutrition.js tápanyagszámítás alapja, és az élelmiszer-adatbázisból való közvetlen naplózáshoz is ez szolgál forrásként.

comments – Polimorf megjegyzés

A comments tábla receptekhez és cikkekhez egyaránt kapcsolódhat: a recipe_id és az article_id mezők közül mindig pontosan az egyik tartalmaz értéket, a másik NULL. Ez a polimorf asszociáció elkerüli egy külön recipe_comments tábla szükségességét.

6.3 Kapcsolatrendszer

Minden felhasználóhoz kapcsolódó rekord kaszkádosan törlődik, ha a felhasználói fiók megszűnik (ON DELETE CASCADE). A recipe diet_type_id mezője SET NULL-ra vált, ha a diétatípus törlődik, megőrizve ezzel a recept adatait. A ratings táblán UNIQUE(user_id, recipe_id) feltétel biztosítja, hogy felhasználónként csak egy értékelés létezzen receptenként – a frissítés INSERT ... ON DUPLICATE KEY UPDATE logikával működik.

A snapshot-alapú meal_items megközelítés az adatmodell egyik legfontosabb mérnöki döntése. A meal_items tábla közvetlenül tárolja a bejegyzés pillanatában érvényes kalória- és makróértékeket, nem hivatkozik vissza receptre idegen kulccsal. Ha a felhasználó rögzít egy étkezést egy adott recept alapján, majd a recept szerzője módosítja az összetevőket, a régi naplóbejegyzés változatlan marad. Ez az étkezési naplóban alapvető elvárás: a historikus adatok integritása nem sérülhet. A denormalizáció ára (redundáns adattárolás) az MVP méreténél messze megéri a historikus pontosságot.

Konkrét adatfolyam-példa: a felhasználó naplóz egy Csirkesalata receptet (2 adag). A `meals.service.js` meghívja a `calculateRecipeNutrition()` függvényt, amely kiszámítja az adagonkénti makrókat (305,6 kcal, 28g fehérje, 5g szénhidrát, 18g zsír). Ezt megszorozza az adagszámmal (x2), majd `meal_items.INSERT` hívással menti a pontos értékekkel. A dashboard `GET /api/meals?date=YYYY-MM-DD` hívásra JOIN-olja a táblákat, összesíti a napi bevitelt, és összehasonlítja a `calorie_target` értékével. Ha a napi cél 2259 kcal és a naplózott összeg 1850 kcal, a dashboard 409 kcal maradékot jelenít meg.

6.4 Adatfolyamok

Étkezés naplózása receptből

A felhasználó kiválaszt egy receptet és megad egy adagszámot. A `meals.service.js` lekéri a receptet, meghívja a `calculateRecipeNutrition()` függvényt, kiszámítja az adagonkénti makrókat, megszorozza az adagszámmal, majd `meal_items.INSERT`-tel menti el a naplóba. Ettől a ponttól a bejegyzés független a forrásrecepttől.

Napi összesítő lekérdezése

A dashboard a `GET /api/meals?date=YYYY-MM-DD` hívásra a `meals` és `meal_items` táblákat JOIN-olja, összesíti a napi kalória/fehérje/szénhidrát/zsír értékeket, és összehasonlítja a felhasználó `calorie_target` értékével. Ha egy napra nincs naplóbejegyzés, az összesítő nullákkal tér vissza – nem hibával.

```
1 async function findDailyTotals(userId, fromDate, toDate) {
2   const [rows] = await pool.query(
3     `SELECT
4       DATE_FORMAT(m.meal_date, '%Y-%m-%d') AS date,
5       COALESCE(SUM(mi.calories), 0) AS calories,
6       COALESCE(SUM(mi.protein), 0) AS protein,
7       COALESCE(SUM(mi.carbs), 0) AS carbs,
8       COALESCE(SUM(mi.fat), 0) AS fat
9     FROM meals m
10    LEFT JOIN meal_items mi ON m.id = mi.meal_id
11   WHERE m.user_id = ? AND m.meal_date BETWEEN ? AND ?
12   GROUP BY m.meal_date
13   ORDER BY m.meal_date ASC`,
14     [userId, fromDate, toDate]
15   );
16   return rows;
17 }
```

6.2. ábra: Napi tápanyag-összesítő SQL lekérdezés.

7. Megvalósítás

7.1 Moduláris felépítés

A backend kódszerkezése domain-alapú: minden üzleti területhez külön route, controller, service és – ahol szükséges – model fájl tartozik. A utils/ könyvtár három pure function-alapú segédmodult tartalmaz: `tdeeCalculator.js` (TDEE-számítás), `recipeNutrition.js` (tápanyagszámítás) és `progressCalculator.js` (kalóriaegyenleg, adherencia, becsült testtömeg-változás). Ezek pure function-ok: azonos bemenetre mindig azonos kimenetet adnak, és könnyen tesztelhetők.

7.2 TDEE-számítás – frontend-backend tükrözés

A TDEE-számítás logikája mind a backenden, mind a fronton implementálva van. Ez azonnali visszajelzést tesz lehetővé a profiloldalon: a felhasználó valós időben látja a kalóriaajánlást, ahogy módosítja az adatait – API-hívás nélkül. A hiteles számítást minden esetben a backend végzi; a frontend-oldali számítás csak előnézet.

A számítás három lépésből áll: először a BMR (alapanyagcsere) kiszámítása nemenként a Mifflin–St Jeor-formula alapján (férfi: $10 \times \text{kg} + 6,25 \times \text{cm} - 5 \times \text{év} + 5$; nő: $10 \times \text{kg} + 6,25 \times \text{cm} - 5 \times \text{év} - 161$), majd a $\text{TDEE} = \text{BMR} \times \text{aktivitási szorzó}$ (sedentary: 1,2; lightly_active: 1,375; moderately_active: 1,55; very_active: 1,725; extra_active: 1,9), végül a célkorrekció és a biztonsági minimum alkalmazása.

7.2.1 A TDEE-számítás - implementáció

```
1 function calculateRecommendedCalories(userDetails, goal) {
2   const { weight, height, age, sex, activity_level } = userDetails || {};
3
4   const w = Number(weight);
5   const h = Number(height);
6   const a = Number(age);
7
8   // All four biometric inputs are required for Mifflin-St Jeor
9   if (!w || !h || !a || w <= 0 || h <= 0 || a <= 0) return null;
10  if (sex !== 'male' && sex !== 'female') return null;
11  if (!ACTIVITY_MULTIPLIERS[activity_level]) return null;
12
13  // Mifflin-St Jeor BMR
14  // Male: 10w + 6.25h - 5a + 5
15  // Female: 10w + 6.25h - 5a - 161
16  const sexOffset = sex === 'male' ? 5 : -161;
17  const bmr = Math.round(10 * w + 6.25 * h - 5 * a + sexOffset);
18
19  const multiplier = ACTIVITY_MULTIPLIERS[activity_level];
20  const tdee = Math.round(bmr * multiplier);
21
22  const normalizedGoal = Object.prototype.hasOwnProperty.call(GOAL_ADJUSTMENTS, goal) ? goal : 'maintain';
23  const goalAdjustment = GOAL_ADJUSTMENTS[normalizedGoal];
24  const rawRecommended = Math.round(tdee + goalAdjustment);
25
26  // Clamp to the sex-specific safe minimum
27  const safeMinimum = SAFE_MINIMUMS[sex] ?? SAFE_MINIMUMS.default;
28  const recommendedCalories = Math.max(rawRecommended, safeMinimum);
29  const wasClamped = recommendedCalories !== rawRecommended;
30
31  const goalLabel = GOAL_LABELS[normalizedGoal];
32
33  let explanation =
34    `Your estimated daily energy need is ${tdee} kcal` +
35    (goalAdjustment !== 0
36      ? ` (${goalAdjustment > 0 ? '+' : ''}${goalAdjustment} kcal for ${goalLabel})`
37      : '') +
38    `, giving a recommended target of ${recommendedCalories} kcal/day.`;
39
40  if (wasClamped) {
41    explanation += ` Adjusted to the safe minimum of ${safeMinimum} kcal/day.`;
42  }
43
44  return {
45    bmr,
46    tdee,
47    goalAdjustment,
48    rawRecommended,
49    safeMinimum,
50    wasClamped,
51    recommendedCalories,
52    explanation,
53  };
54 }
```

7.1. ábra: A tápanyagszámítás implementációja.

7.3 Étrendtervező algoritmus

A planner.service.js generateDailyPlan() függvénye a felhasználó profiljából olvassa ki a kalóriaszükségletet és diétatípust, lekérdezi a megfelelő kategóriájú recepteket (ha < 3 egyezés van, az összes receptre visszaesik), Fisher-Yates shuffle-lal véletlenszerűsíti a sorrendet, majd a slot-arányok (0.25/0.35/0.30/0.10) alapján kioszt egy-egy receptet a négy étkezési napszakhoz. Az eredmény nem kerül adatbázisba – minden kéréskor friss terv generálódik.

```
1 async function generateDailyPlan(userId) {
2   const user = await UserModel.findById(userId);
3   if (!user) throw { status: 404, message: 'User not found.' };
4
5   const calorieTarget = user.calorie_target || null;
6   const dietType      = user.diet_type      || null;
7
8   // Try to find recipes matching the user's diet type (stored as category)
9   let pool = dietType ? await RecipeModel.findAll({ category: dietType }) : [];
10
11  // Fall back to all recipes if there aren't enough typed ones
12  if (pool.length < 3) {
13    pool = await RecipeModel.findAll();
14  }
15
16  if (pool.length === 0) {
17    return {
18      calorie_target: calorieTarget,
19      diet_type:      dietType,
20      meals:          { breakfast: null, lunch: null, dinner: null, snack: null },
21      note:            'No recipes found. Add some recipes to generate a plan.',
22    };
23  }
24
25  const picked = shuffle(pool);
26  const slots  = ['breakfast', 'lunch', 'dinner', 'snack'];
27
28  const meals = {};
29  slots.forEach((slot, i) => {
30    const recipe = picked[i] || null;
31    meals[slot] = recipe
32      ? {
33          recipe_id:      recipe.id,
34          title:          recipe.title,
35          category:       recipe.category || null,
36          description:    recipe.description || null,
37          image_url:      recipe.image_url || null,
38          prep_time:      recipe.prep_time || null,
39          servings:       recipe.servings || null,
40          calorie_target: calorieTarget
41            ? Math.round(calorieTarget * SLOT_RATIOS[slot])
42            : null,
43        }
44      : null;
45  });
46
47  return { calorie_target: calorieTarget, diet_type: dietType, meals };
48 }
```

7.2. ábra: Az étrendterv-generáló algoritmus implementációja.

7.4 Bevásárlólista-generálás

A `shopping.service.js` a megadott `recipe_ids` tömbből lekéri a receptek ingredients JSON-tömbjeit, minden sort `parseIngredient()` függvénnyel feldolgoz (AMOUNT_RE reguláris kifejezéssel leválasztja a mennyiséget és egységet), `cleanName()` függvénnyel eltávolítja a leíró szuffixeket (pl. "chicken thighs, bone-in" → "chicken thighs"), `categorise()` kulcsszó-térképpel 7 kategóriába sorolja (Protein, Vegetables, Fruit, Dairy, Grains, Nuts & Seeds, Pantry; egyéb: Other), majd azonos névvel rendelkező hozzávalókat egyetlen csoportba vonja össze. Az eredmény kategória, majd névbetűrend szerint rendezve jelenik meg.

```
1  async function generateFromRecipes(recipeIds) {
2    if (!Array.isArray(recipeIds) || recipeIds.length === 0) {
3      throw { status: 400, message: 'recipe_ids must be a non-empty array.' };
4    }
5
6    const [recipes] = await pool.query(
7      'SELECT id, title, ingredients FROM recipes WHERE id IN (?)',
8      [recipeIds]
9    );
10
11    if (recipes.length === 0) {
12      throw { status: 404, message: 'No matching recipes found.' };
13    }
14
15    // Collect and deduplicate ingredient names
16    const grouped = {}; // name → { name, category, entries: [original strings] }
17
18    for (const recipe of recipes) {
19      const items = parseIngredients(recipe.ingredients);
20      for (const item of items) {
21        if (!item.name) continue;
22        if (!grouped[item.name]) {
23          grouped[item.name] = {
24            name: item.name,
25            category: categorise(item.name),
26            entries: [],
27          };
28        }
29        grouped[item.name].entries.push(item.original);
30      }
31    }
32
33    // Build flat list sorted by category then name
34    const items = Object.values(grouped).sort((a, b) => {
35      const catCmp = a.category.localeCompare(b.category);
36      return catCmp !== 0 ? catCmp : a.name.localeCompare(b.name);
37    });
38
39    const sources = recipes.map((r) => ({ id: r.id, title: r.title }));
40    return { items, sources };
41  }
```

7.3. ábra: A bevásárlólista-generáló algoritmus implementációja.

7.5 Chatbot integráció (AI táplálkozási asszisztens)

Az AI-alapú táplálkozási chatbot opcionális, de szakmailag releváns kiegészítő funkció. A chatbot.service.js az OpenAI gpt-4o-mini modellt használja a Chat Completions API-n keresztül. A felhasználóról átadott kontextus: felhasználónév, étkezési cél, diétatípus, napi kalóriacél és aktuális testtömeg. Adatvédelmi okból a rendszer nem ad át konkrét naplóbejegyzéseket vagy személyazonosításra alkalmas információkat – csak az aggregált profiladatokat.

A rendszer-prompt tartalmaz egy kötelező etikai korlátot: a chatbot tájékoztató jellegű válaszokat adhat, de kifejezetten jelzi, hogy válaszai nem helyettesítik az orvosi vagy dietetikusi szaktanácsot. Hibakezelés: OpenAI API-hiba esetén a szerver 500-as státuszkóddal és érthető hibaüzenettel válaszol; ez nem blokkolja az alkalmazás többi funkcióját. Az API-kulcs kizárólag a szerver oldali .env fájlban tárolódik, a frontend soha nem érintkezik közvetlenül az OpenAI szolgáltatással. Az üzenethistória nem kerül adatbázisba; legfeljebb 10 korábbi üzenet kerül a modell kontextusába.

7.6 REST API végpontok

Az API összesen 40+ végpontot tartalmaz. Az alábbi táblázat a főbb végpontokat foglalja össze.

Végpont	Metódus	Auth	Leírás
/api/auth/register	POST	–	Regisztráció; 201 vagy 400/409
/api/auth/login	POST	–	Bejelentkezés; token visszaadva
/api/users/me	GET/PUT	Bearer	Profil lekérése és frissítése
/api/recipes	GET	–	Receptlista; ?search=...&category=...
/api/recipes/:id/nutrition	GET	–	Futásidejű tápanyagszámítás
/api/recipes	POST	Bearer	Recept létrehozása
/api/recipes/:id	PUT/DELETE	Bearer	Módosítás/törlés (csak saját)
/api/meals	GET	Bearer	Napi napló; ?date=YYYY-MM-DD
/api/meals/history	GET	Bearer	Haladástörténet; ?days=7
/api/meals/log-recipe	POST	Bearer	Recept közvetlen naplózása
/api/meals/:id/items/from-food	POST	Bearer	Élelmiszer-adatbázisból naplózás
/api/planner/daily	GET	Bearer	Napi étrendterv generálása
/api/shopping/generate	POST	Bearer	Bevásárlólista generálása receptekből
/api/chatbot/message	POST	Bearer	AI chatbot üzenetküldés
/api/health	GET	–	Health-check; 200 {status:'ok'}

7.4. ábra: REST API főbb végpontjai.

7.7 Hibakezelés és validáció

A service-függvények strukturált hibaobjektumokat dobnak (`{ status, message }`), amelyeket a controllerek egységes catch-blokkban kezelnek. Ez garantálja, hogy belső hibainformáció (stack trace) soha ne szivároгjon a klienshez, és minden hibás kérés `{ error: '...' }` formátumú JSON-választ kapjon.

A validáció három szinten valósul meg: kliensoldali ellenőrzés (UX-javítás, azonnali visszajelzés), szerver oldali controller szintű ellenőrzés (kötelező mezők jelenléte, engedélyezett mezőlista – pl. password_hash és role kizárása profilfrissítéskor), és szerver oldali service szintű ellenőrzés (tulajdonosi jogosultság, ENUM-értékkészlet, numerikus tartomány). Adatbázis szinten UNIQUE, NOT NULL, CHECK és FK CASCADE kényszerfeltételek egészítik ki ezeket.

8. Felhasználói felület és UX

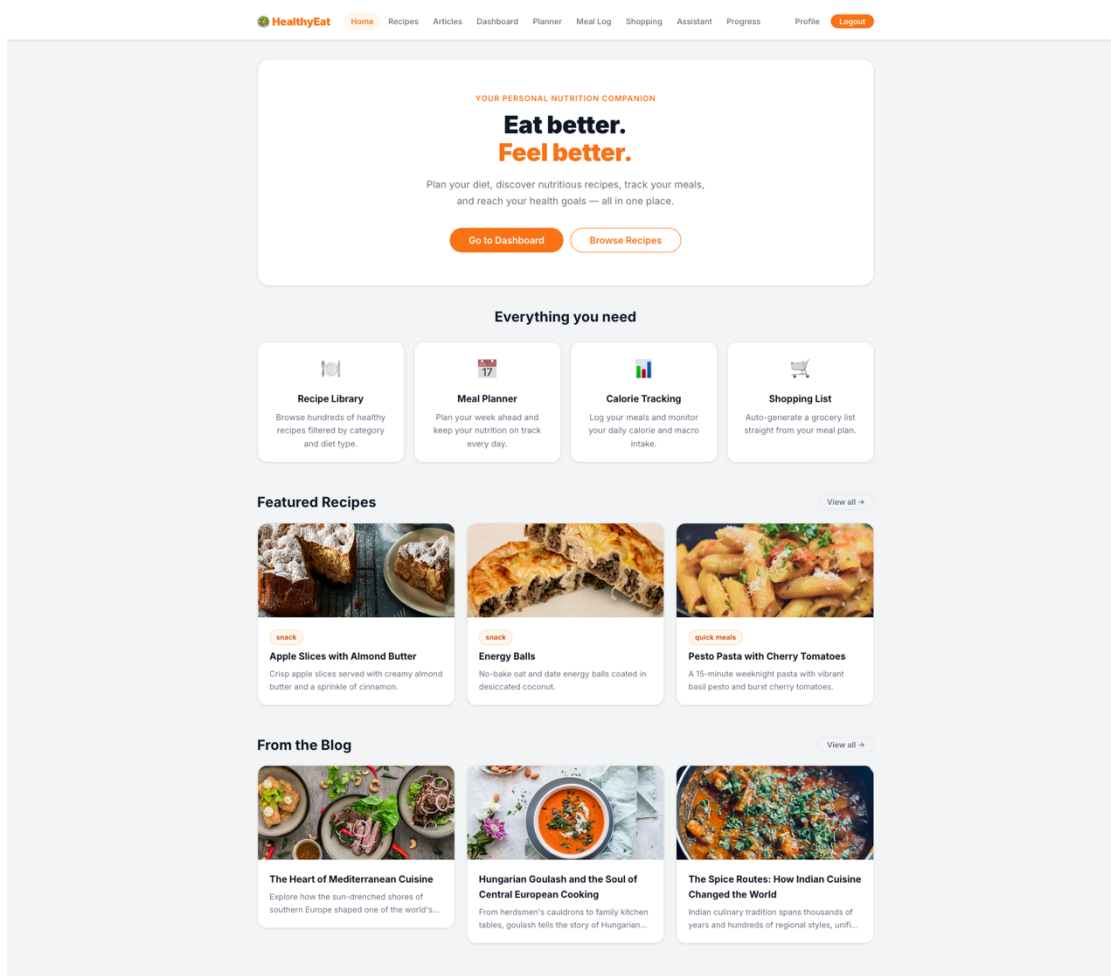
8.1 Navigációs struktúra

Az alkalmazás 19 oldalkomponensből áll. A navigációs sáv (Navbar.jsx) bejelentkezett és vendég állapotban eltérő menüpontokat jelenít meg. Mobilnézetben (< 768 px) hamburger-ikon és kinyíló menü, asztali nézetben vízszintes navigáció. Az ImgWithFallback komponens kezeli a hiányzó receptképeket – hibás forrás esetén automatikusan helyettesítő képre vált.

8.2 Főbb oldalak

Nyitóoldal (HomePage)

A nyitóoldal az alkalmazás belépési pontja, hitelesítés nélkül is elérhető. Bemutató szöveg és kiemelő kártyák kommunikálják az értékajánlatot – étrendtervező, étkezésnapló, haladáskövetés, receptgyűjtemény –, prominens regisztrációs gombbal. Az oldal alján néhány kiemelt recept ösztönzi a vendéget a tartalom felfedezésére.



8.1. ábra: Nyitóoldal.

Profiloldal (ProfilePage)

A profiloldal két részre tagolódik: felül az alapadatok (felhasználónév, e-mail), alul a TDEE-paraméterek szerkeszthető formja. Adatbevitel közben a rendszer valós időben mutatja a kalóriaajánlást (frontend-oldali számítás). Mentés után egy kiemelő panelen jelenik meg a BMR, TDEE és a végső kalóriacél, valamint – ha a számított érték a biztonsági minimum alá esne – egy figyelmeztető üzenet.

The screenshot shows the 'Profile' page of the HealthyEat application. The page has a light gray background and a white content area. At the top, there is a navigation bar with the 'HealthyEat' logo and links to Home, Recipes, Articles, Dashboard, Planner, Meal Log, Shopping, Assistant, Progress, Profile, and Logout. The 'Profile' section is titled 'Profile' with the subtitle 'Manage your personal information and nutrition goals'. It contains four main sections: 'Account' with fields for Username (user) and Email (user@test.com, with a note 'Email cannot be changed'); 'Body Metrics' with fields for Weight (kg, e.g. 75), Height (cm, e.g. 175), Age (years, e.g. 28), Biological sex (dropdown), and Activity level (dropdown); 'Nutrition Goals' with a Goal dropdown, Daily calorie target (kcal, e.g. 2000, with a note 'You can accept the recommendation above or enter your own value.'), and Diet type (e.g. vegan, keto, balanced); and 'Notification Preferences' with a Preferences (JSON) field (e.g. {"email": true, "push": false}). A 'Save changes' button is at the bottom.

8.2. ábra: Profil oldal.

Recept adatlap (RecipeDetailPage)

A recept adatlapján borítókép, összefoglaló adatok (elkészítési idő, adagszám, kategória), hozzávalólista, elkészítési lépések és egy tápanyag-összesítő panel kapnak helyet. Bejelentkezett felhasználóknak értékelési widget (1–5 csillag), komment szekció és gyorsgombok (Étrendtervbe ad, Étkezésbe rögzít) is megjelennek.

The screenshot shows the 'Cottage Cheese Pancakes' recipe page in the HealthyEat app. The page layout includes a top navigation bar with links like Home, Recipes, Articles, Dashboard, Planner, Meal Log, Shopping, Assistant, Progress, Profile, and Logout. The main content area features a large image of the pancakes, a 'high-protein' tag, the recipe title, a brief description, and a detailed 'Instructions' section. To the right, there's a 'Meal Log' button, a 'Nutrition' panel showing 234.5 kcal, 15.1g protein, 29.7g carbs, and 6.4g fat, and a 'Recipe Info' panel with a 20-minute prep time and 2 servings. Below these is a 'Rating' section with five stars and an 'Ingredients' list. At the bottom, there's a 'Comments (0)' section with a text input field and a 'Post comment' button. The footer of the page states 'No comments yet. Be the first!'.

HealthyEat Home Recipes Articles Dashboard Planner Meal Log Shopping Assistant Progress Profile Logout

← Back to recipes

Cottage Cheese Pancakes

High-protein fluffy pancakes made with cottage cheese — under 300 calories a serving.

Instructions

Blend cottage cheese, eggs, oat flour, baking powder, honey and vanilla until smooth.

Heat coconut oil in a non-stick pan over medium-low heat.

Pour small rounds; cook 2–3 minutes until bubbles form, then flip and cook 1 minute.

Serve with a dollop of Greek yogurt and fresh berries.

Meal Log

+ Log this recipe

Nutrition per serving

- Calories 234.5 kcal
- Protein 15.1 g
- Carbs 29.7 g
- Fat 6.4 g

* Based on 3 matched ingredients. 5 could not be matched.

Recipe Info

- Prep time 20 min
- Servings 2
- Category high-protein

Rating

No ratings yet.

★ ★ ★ ★ ★

Ingredients

- 200g low-fat cottage cheese
- 3 eggs
- 60g oat flour
- 1 tsp baking powder
- 1 tbsp honey
- 1 tsp vanilla extract
- coconut oil for frying
- Greek yogurt and berries to serve

Comments (0)

Write a comment...

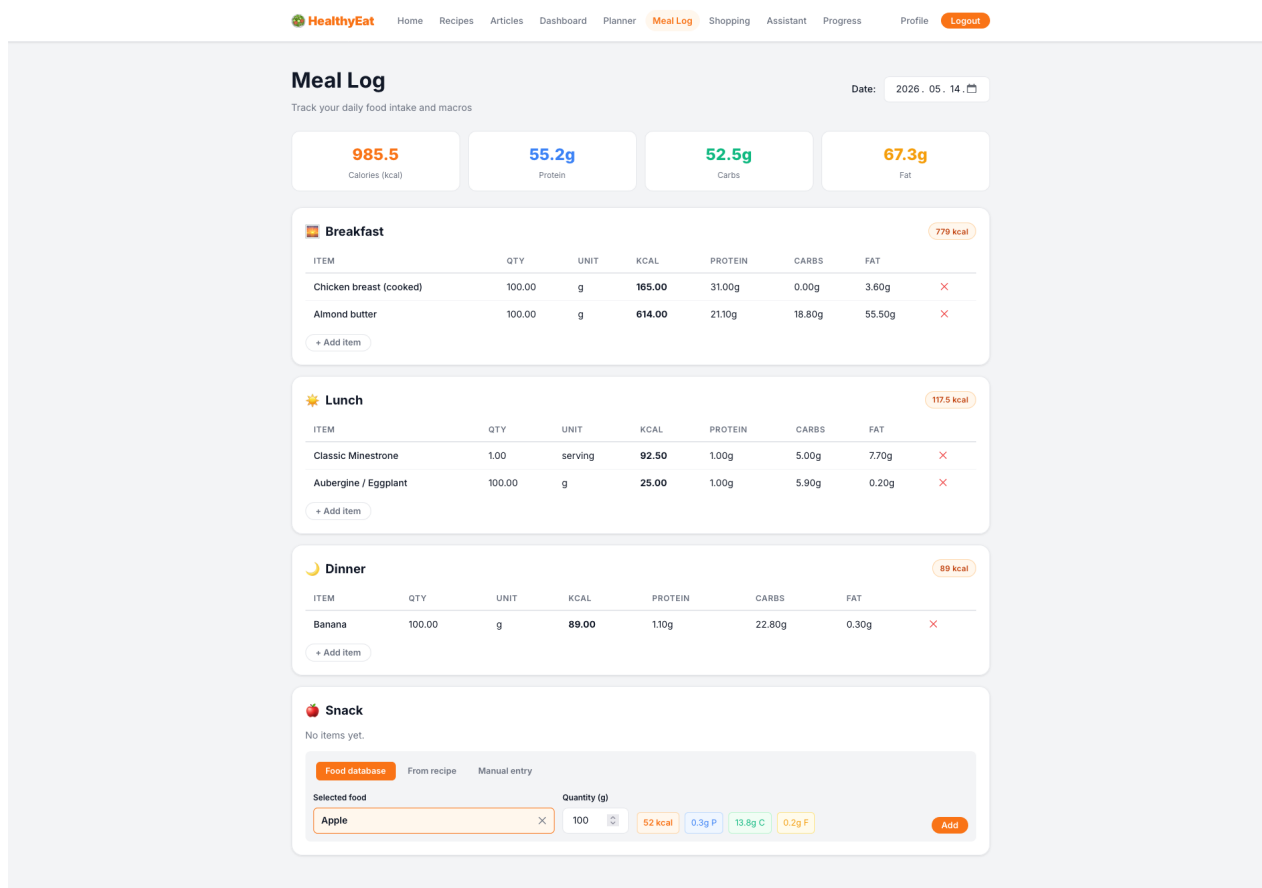
Post comment

No comments yet. Be the first!

8.3. ábra: Recept adatlap oldal.

Étkezésnapló (MealLogPage)

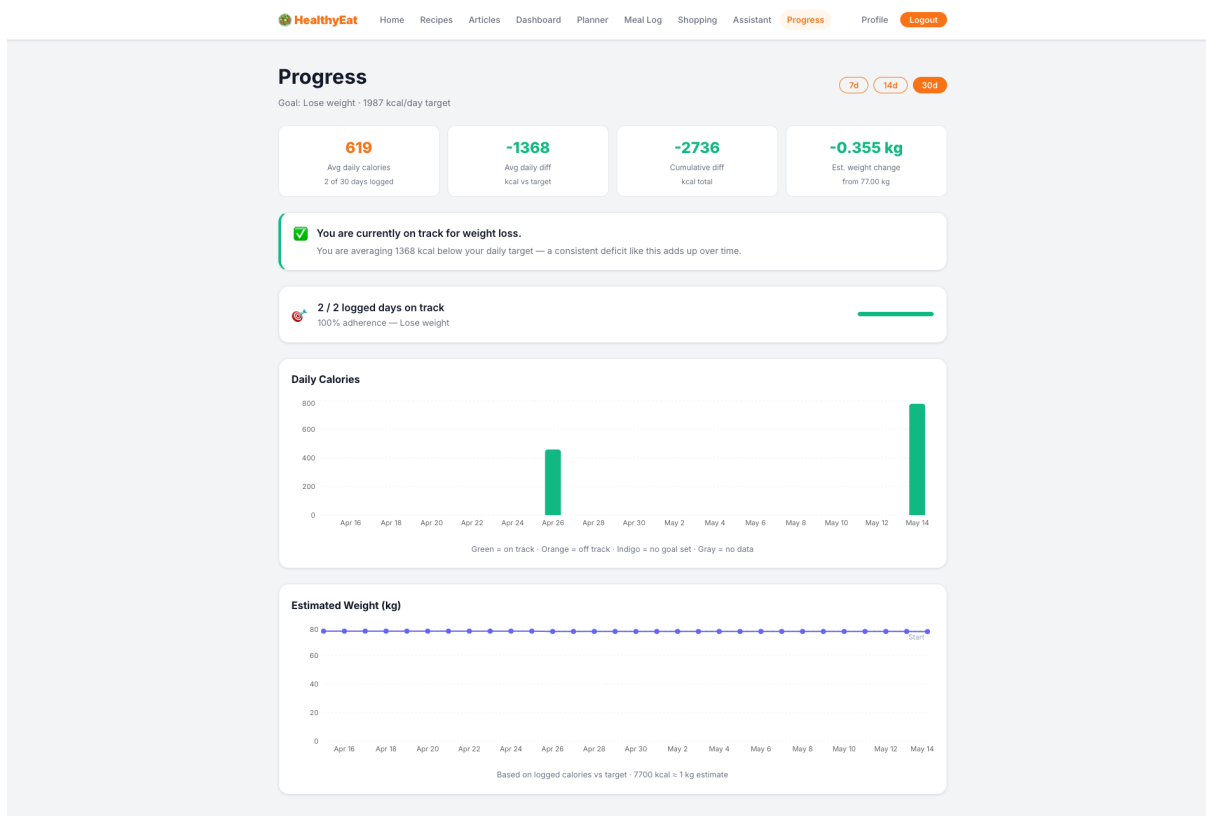
Az oldal tetején dátumválasztó, alatta a kiválasztott nap étkezési bejegyzései étkezési típusonként csoportosítva (összecsukható szekciókban). Minden tételnél neve, mennyisége, kalóriája és makrói láthatók; szerkesztés és törlés gombokkal. Felül összesítő progress-bar a napi kalóriabevitelről. Új tétel modális párbeszédpanelen keresztül adható hozzá.



8.4. ábra: Étkezésnapló oldal.

Haladáskövetés (ProgressPage)

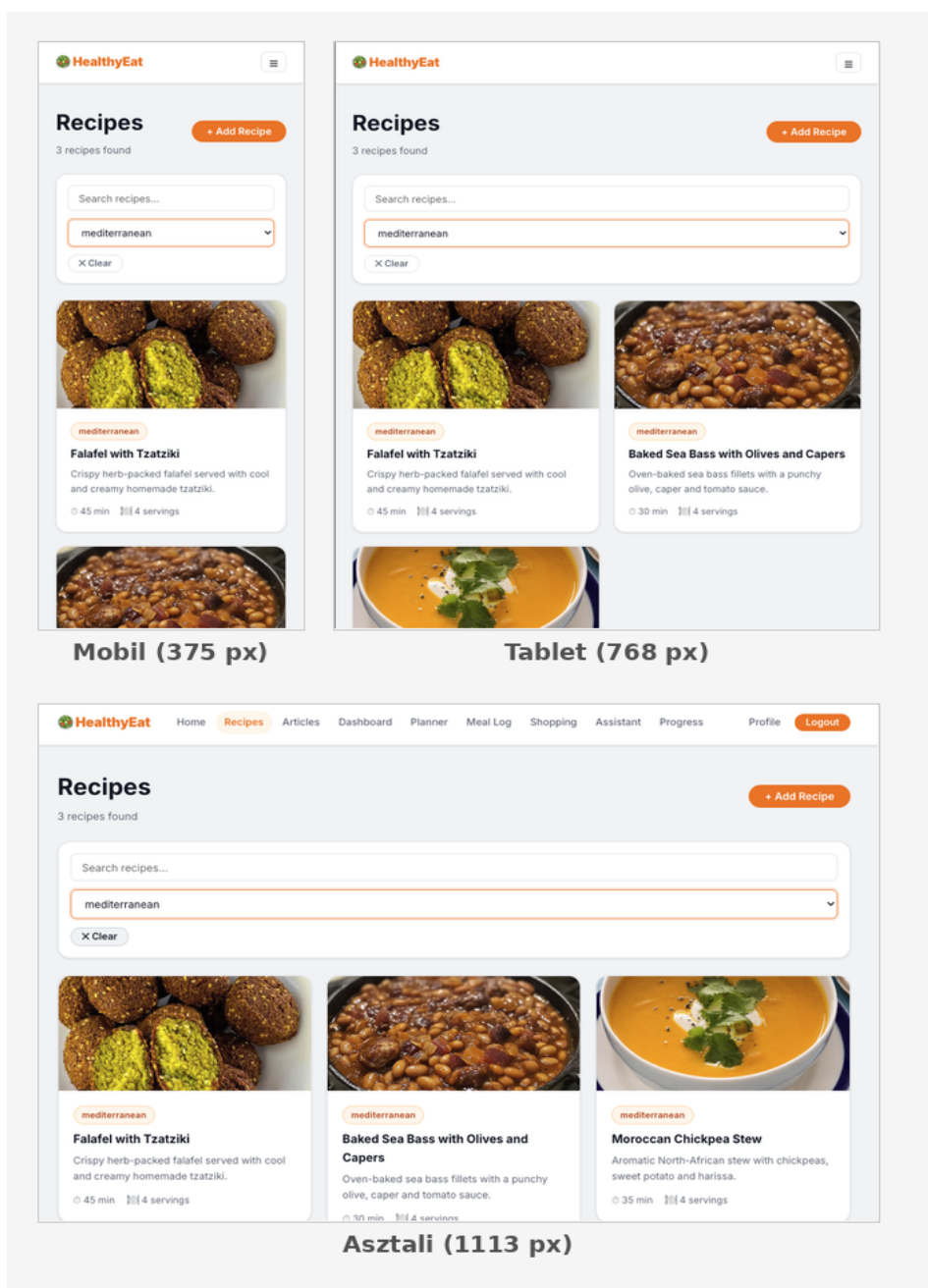
Dátumtartomány-választó (alapértelmezett: elmúlt 7 nap), alatta Recharts vonalas és sávdíagramok: napi kalóriabevitel vs. kalóriacél (vízszintes referenciavonal), makrók idősoros alakulása. Összesítő statisztikák: átlagos napi kalória, célteljesítési arány (adherencia %). A ResponsiveContainer biztosítja, hogy a grafikonok mobil nézetben is olvashatók maradjanak.



8.5. ábra: Haladáskövetés oldal.

8.3 Reszponzív design

Az alkalmazás reszponzivitása CSS-alapú, komponensszintű médiaquery-kkel valósul meg, dedikált UI-könyvtár (pl. Material UI, Tailwind) nélkül. Három törési pontot definiál: mobilon (≥ 375 px) egyoszlopos elrendezés hamburger navigációval; tableten (≥ 768 px) kétszlopos receptkártyák; asztali gépen (≥ 1024 px) teljes vízszintes navigáció és többszlopos tartalom.



8.6. ábra: A RecipesPage megjelenése különböző képernyőméreteken.

8.4 Hibás és üres állapotok

Minden üres lista, hibaállapot és sikeres művelet egyértelmű visszajelzéssel jár. Néhány konkrét példa: ha nincs naplóbejegyzés az adott napra, a dashboard nem üresen jelenik meg, hanem tájékoztató szöveggel és gyorsgombbal; üres receptadatbázis esetén az étrendtervező note mezőben jelzi, hogy nincs recept; hiányzó receptkép helyén az ImgWithFallback komponens helyettesítő képet mutat; a chatbot oldal fejlécén fix sávban megjelenik az etikai nyilatkozat (a chatbot válaszai tájékoztató jellegűek, nem helyettesítik az orvosi szaktanácsot).

9. Tesztelés és validáció

9.1 Tesztelési stratégia

A rendszer tesztelése kézi funkcionális tesztelésen alapul – automatizált tesztelési keretrendszer (Jest, Supertest) bevezetése nem fért az MVP fejlesztési keretébe. A tesztelés négy területre terjedt ki: API-végpontok HTTP-kérésekkel a frontendből és Postmanból, teljes munkafolyamatok böngészős végigkövetése, egységszintű logika ellenőrzése ismert bemeneti értékekkel (tdeeCalculator.js, recipeNutrition.js, progressCalculator.js), és határesetek szándékosan hibás bemenetekkel.

9.2 Funkcionális tesztesetek

A 25 teszteset lefedi a rendszer összes fő use case-ét: autentikáció (TC-01–06), profil és TDEE (TC-07–08), receptek (TC-09–12), étkezésnapló (TC-13–16), étrendtervező (TC-17–18), bevásárlólista (TC-19–21), haladáskövetés (TC-22), cikkek és értékelés (TC-23–25). Minden teszteset sikeres eredménnyel zárult.

ID	Modul	Típus	Rövid leírás	Eredmény
TC-01	Auth	Funk.	Regisztráció érvényes adatokkal → 201, password_hash nélkül	✓
TC-02	Auth	Funk.	Bejelentkezés → 200 + token	✓
TC-03	Auth	Neg.	Helytelen jelszó → 401	✓
TC-04	Auth	Határ	Duplikált e-mail → 409	✓
TC-05–06	Auth	Neg.	Token nélkül/lejárt tokennel → 401	✓
TC-07–08	Profil	Funk.	Profil mentése → TDEE kiszámítva (2259 kcal, ld. 9.3)	✓
TC-09–12	Receptek	Funk./Neg.	Lista, létrehozás, más user tokennel 403, tápanyagszámítás	✓
TC-13–16	Napló	Funk./Neg.	Receptnaplózás snapshot-makrókkal, nem létező élelmiszer 404, qty=0 → 400	✓

TC-17-18	Tervező	Funk./Határ	4 slottal rendelkező terv; üres adatbázis → note mező	✓
TC-19-21	Bevásárlólista	Funk./Neg./UI	Kategorizált lista; üres recipe_ids → 400; localStorage-megőrzés	✓
TC-22	Haladás	Funk.	7 napos tömb onTrack mezőkkel	✓
TC-23-25	Auth/Komment/Értékelés	Auth/Funk.	Cikk user tokennel → 403; komment létrehozás; dupla értékelés → UPDATE	✓

9.1. ábra: Funkcionális tesztesetek összefoglalója

9.3 TDEE-kalkulátor numerikus verifikációja

A calculateRecommendedCalories() pure function kézi ellenőrzése két konkrét referencia-esettel:

Normál eset – férfi, fogyási cél

Bemenet: 80 kg, 180 cm, 30 év, férfi, mérsékelten aktív, fogyási cél. $BMR = 10 \times 80 + 6,25 \times 180 - 5 \times 30 + 5 = 1780$ kcal. $TDEE = 1780 \times 1,55 = 2759$ kcal. Céلكorrekción: -500 kcal. Végeredmény: 2259 kcal. Biztonsági minimum (1500 kcal) nem aktiválódik. A kalkulátor kimenete egyezik a kézi számítással ✓.

Minimumszint-korlátozás – nő, nagyon alacsony bevitel

Bemenet: 45 kg, 155 cm, 20 év, nő, ülő életmód, fogyási cél. $BMR = 1158$ kcal. $TDEE = 1158 \times 1,2 = 1390$ kcal. Céلكorrekción: -500 kcal → 890 kcal. Biztonsági minimum (1200 kcal) aktiválódik; a kalkulátor 1200 kcal-t ad vissza (wasClamped: true). A tényleges kimenet egyezik a várttal ✓.

9.4 Tápanyagszámítás tesztelése

Teszt recept: "Csirkesalata" – ["200g chicken breast", "100g cucumber", "2 tbsp olive oil"], 2 adag. Kézi számítás: 200g csirke (165 kcal/100g) = 330 kcal; 100g uborka (16 kcal/100g) = 16 kcal; 30g olívaolaj (884 kcal/100g) = 265,2 kcal. Összesen: 611,2 kcal / 2 adag = 305,6 kcal/adag. A recipeNutrition.js kimenete egyezik a kézi számítással ✓.

9.5 Jogosultságkezelés tesztelése

A három middleware-típus mindegyike külön tesztelési forgatókönyvön esett át: token nélküli kérés → 401; érvénytelen token → 401; saját erőforrás módosítása → 200; más user erőforrása → 403; admin végpont user tokennel → 403; admin végpont admin tokennel → 201; opcionális auth token nélkül → 200 (userScore: null); opcionális auth tokennel → 200 (userScore: N). Minden eset a várt eredménnyel zárult ✓.

9.6 A tesztelés korlátai

Az automatizált tesztsuite hiánya az MVP legnagyobb technikai adóssága: kódmódosítás utáni regressziót csak ismételt kézi teszteléssel lehet detektálni. A `tdeeCalculator.js` és a `recipeNutrition.js` pure function-ok jest tesztekkel könnyen lefedhetők lennének – ez a legprioritásabb jövőbeli fejlesztési lépés. Az admin modul controller-implementációja TODO-jelzéssel rendelkezik; az `admin.routes.js`-ből hiányzó `authenticate + authorize` middleware-lánc biztonsági kockázatot jelent, amelyet implementálás előtt pótolni szükséges. Terhelési és teljesítménytesztelés, böngészőkompatibilitás-vizsgálat (csak Chrome-on tesztelve) és penetrációs tesztelés nem volt elvégezve.

9.7 Korlátok és kockázatok

Automatizált tesztsuite hiánya: a rendszer nem rendelkezik unit tesztekkel (Jest) vagy integrációs tesztkészlettel. Kódmódosítás utáni regressziót csak ismételt kézi teszteléssel lehet detektálni; ez a skálázhatóság és a hosszú távú karbantarthatóság szempontjából a legnagyobb technikai adósság. A `tdeeCalculator.js` és a `recipeNutrition.js` pure function-ok jest tesztekkel könnyen lefedhetők lennének.

Admin modul korlátozott állapota: az `admin.routes.js`-ből hiányzik az `authenticate + authorize('admin')` middleware-lánc, ami azt jelenti, hogy az admin végpontok jelenleg nem megfelelően védettek.

JWT localStorage-tárolás: a token JavaScript-ből olvasható, ami XSS-támadás esetén tokenlopást tesz lehetővé. Biztonságosabb alternatíva a `httpOnly` cookie + CSRF-token kombináció.

Egészségügyi felelősségi keret: az alkalmazás egészségközeli adatokat kezel. A chatbot felületen etikai nyilatkozat tájékoztatja a felhasználót, hogy a rendszer válaszai nem helyettesítik az orvosi szaktanácsot. Az alkalmazás nem kezel orvosi diagnózisokat és nem ad gyógyszerészeti tanácsokat.

10. Összefoglalás és jövőbeli fejlesztések

10.1 Elért eredmények

A HealthyEat webalkalmazás megvalósítja az MVP-definícióban meghatározott összes kötelező funkciót. A rendszer egy végigvihető, koherens felhasználói folyamatot biztosít: a profilbeállítástól és a TDEE-számítástól a receptböngészésen és az étrendterv generálásán át az étkezésnapló vezetéséig, a bevásárlólista automatikus összeállításáig és a napi/heti haladás vizualizálásáig.

A kézi funkcionális tesztesetek 100%-os sikerességgel zárultak. A TDEE-kalkulátor numerikus verifikációja megerősítette az 1 kcal alatti eltérést. Az MVP koherenciájának bizonyítéka az adatmodell szintjén is megragadható: a `users.calorie_target` a tervező algoritmus alapértéke; a `recipes` tábla mind a tervező, mind a bevásárlólista forrása; a `meal_items` tábla adja a dashboard és a haladáskövetés bemenetét – minden modul a szomszédos modul kimenetét veszi bemenetként.

10.2 Fejlesztési tanulságok

A fejlesztés során több olyan döntés született, amelyek utólag a legjobb választásnak bizonyultak. A snapshot-alapú `meal_items` megközelítés (ADR-005) a denormalizáció árán historikus adatintegritást biztosít – ez az étkezési napló alapvető elvárása. A TDEE-számítás frontend-backend tükrözése azonnali felhasználói visszajelzést tesz lehetővé felesleges API-hívások nélkül. A migrációalapú sémafejlesztés lehetővé tette az iteratív munkát anélkül, hogy az alapsémát kellett volna folyamatosan módosítani.

Ahol viszont máshogy döntöttem volna: az admin modul controller-implementációja TODO-jelzéssel maradt, ami biztonsági kockázatot jelent. Az automatizált tesztsuite hiánya is korai befektetésként lett volna érdemes – a pure function-ok (`tdeeCalculator`, `progressCalculator`) utólag is könnyen tesztelhetők, de jobb lett volna ezt a fejlesztéssel párhuzamosan felépíteni.

10.3 Jövőbeli bővítési lehetőségek

Rövid távon a legfontosabb fejlesztési irányok: automatizált tesztsuite (Jest + Supertest) a pure function-okhoz és az API-végpontokhoz; az admin modul teljes implementációja a hiányzó middleware-lánccal; az allergia- és intoleranciaszűrő bevezetése (recept szintjén egy allergens mező + felhasználói szűrőlista).

Középtávon a heti/havi étrendterv-ütemezés (`weekly_plans` tábla) és a makróarány-alapú tervező (jelenleg csak kalóriaarányt optimalizál) lennének logikus következő lépések. Hosszabb távon egy mobil natív alkalmazás (React Native, az Express REST API változatlanul felhasználható) és fitness API-integráció (Garmin, Fitbit – az aktivitási szint automatikus importálása pontosítaná a TDEE-számítást) egészítenék ki a rendszert.

Irodalomjegyzék

- [1] Magyar Táplálkozástudományi Társaság (2022). Táplálkozási ajánlások. Budapest.
- [2] Hollis, J. F. et al. (2008). Weight loss during the intensive intervention phase of the weight-loss maintenance trial. *American Journal of Preventive Medicine*, 35(2), 118–126.
- [3] Mifflin, M. D. et al. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241–247.
- [4] Under Armour Inc. (2022). MyFitnessPal: Platform overview. <https://www.myfitnesspal.com> [Utolsó megtekintés: 2026. 05. 01.]
- [5] YAZIO GmbH (2024). Yazio: Calorie counter and nutrition tracker. <https://www.yazio.com> [Utolsó megtekintés: 2026. 05. 01.]
- [6] Lifesum AB (2024). Lifesum: Healthy eating and diet app. <https://lifesum.com> [Utolsó megtekintés: 2026. 05. 01.]
- [7] Mealime Inc. (2024). Mealime: Meal planner and recipes. <https://www.mealime.com> [Utolsó megtekintés: 2026. 05. 01.]
- [8] React (2024). React – A JavaScript library for building user interfaces. <https://react.dev> [Utolsó megtekintés: 2026. 05. 01.]
- [9] Vite (2024). Vite – Next generation frontend tooling. <https://vitejs.dev> [Utolsó megtekintés: 2026. 05. 01.]
- [10] Express.js (2024). Express – Fast, unopinionated web framework for Node.js. <https://expressjs.com> [Utolsó megtekintés: 2026. 05. 01.]
- [11] MySQL (2024). MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc> [Utolsó megtekintés: 2026. 05. 01.]
- [12] OpenAI (2024). OpenAI API Reference. <https://platform.openai.com/docs> [Utolsó megtekintés: 2026. 05. 01.]
- [13] Recharts (2024). Recharts – A composable charting library. <https://recharts.org> [Utolsó megtekintés: 2026. 05. 01.]

Nyilatkozatok

Nyilatkozat

Alulírott Bátai Zsolt programtervező informatikus szakos hallgató kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Mesterséges intelligencia használatának nyilatkozata

A szakdolgozat készítése során mesterséges intelligencia alapú eszközöket vettem igénybe a szoftverfejlesztési és dokumentációs munkafolyamat egyes lépéseinek támogatásához. Kijelentem, hogy az MI által javasolt tartalmakat minden esetben személyesen ellenőriztem és értékeltem. A dolgozatban szereplő tervezési döntések, architekturális választások és mérnöki következtetések a saját elemzőmunkám eredményei.

Alkalmazott eszközök

A szakdolgozat elkészítése során két generatív mesterséges intelligencia alapú eszközt vettem igénybe: az Claude Code környezetet (Claude Sonnet 4.6 modell), valamint a ChatGPT-t (OpenAI GPT-5.5 modell).

A Claude Code-ot elsősorban a forráskód elemzésére, a rendszertervezési dokumentációk és PlantUML diagramok elkészítésének támogatására, valamint a kódbázis felépítésének áttekintésére használtam. A ChatGPT-t a szakdolgozat szövegének stilisztikai és nyelvi javítására, a megfogalmazások pontosítására, valamint a dokumentum szerkesztésével kapcsolatos kérdések tisztázására alkalmaztam.

A rendszer architektúrájának megtervezése, a technológiai döntések meghozatala és az alkalmazás megvalósítása teljes egészében saját munkám eredménye.

2026. május



Aláírás

Elektronikus mellékletek

A webalkalmazás GitHub elérhetősége: <https://github.com/h255327/szakdolgozat>

Végleges branch: main

Végleges commit azonosító: 1e3e559

Futtatás: részletes telepítési útmutató a GitHub-adattár README.md fájljában érhető el.